

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Аппаратное и программное обеспечение сетей»

Тема: «Организация UDP – сервера и UDP - клиента»

Выполнил:

Студент 3 курса

Группы ИИ-27

Литовкин Н. А.

Проверила:

Кулик А. Д.

Брест 2026

Цель работы: Приобрести навыки по практическому использованию библиотеки для реализации сетевых приложений на базе протоколов TCP и UDP.

Ход работы:

Вариант 12

Задание на выполнение

10.1 Изучить теоретический материал, функции и классы пакета java.net и листинг программы реализации UDP-сервера и UDP-клиента. Получить индивидуальное задание у преподавателя.

10.2 Разработать программу работы UDP-сервера, выполняющую функции согласно варианта задания к лабораторной работе 1 (см. приложения).

10.3 Разработать программу работы UDP-клиента (см. приложения).

10.4 Выполнить проверку программ UDP – сервера и UDP – клиента, используя адрес 127.0.0.1 и порт 666. Продемонстрировать работу системы преподавателю.

10.5 Представить отчет, содержащий титульный лист, листинг программы с подробными комментариями основных фрагментов программ.

10.6 Подготовиться к защите лабораторной работы по теоретическому материалу, функциям, собственным результатам, полученным в ходе выполнения лабораторной работы.

Задание для реализации UDP-сервера:

После приема каждой группы из 64 символов от клиента сервер формирует и отправляет клиенту цепочку <«символ1-количество1», «символ2-количество2» и т.д.>, где «символ» - встреченный в группе символ, а «количество» - их количество в группе. Если в очередной группе число разных символов меньше 3, то отправляется соответствующее сообщение клиенту и сервер разрывает соединение.

Код программы

```
#include <iostream>
#include <string>
#include <map>
#include <cstring>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/select.h>
#include <unistd.h>

using namespace std;

const int PORT = 666;
const int BUFFER_SIZE = 1024;

int main() {
    int serverSocket = socket(AF_INET, SOCK_DGRAM, 0);

    sockaddr_in serverAddr{};
```

```

serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

bind(serverSocket,
    reinterpret_cast<sockaddr*>(&serverAddr),
    sizeof(serverAddr));

cout << "UDP server started on port " << PORT << endl;

while (true) {
    fd_set readSet;
    FD_ZERO(&readSet);

    FD_SET(serverSocket, &readSet);
    FD_SET(STDIN_FILENO, &readSet);

    int maxFd = serverSocket;

    select(maxFd + 1, &readSet, nullptr, nullptr, nullptr);

    if (FD_ISSET(serverSocket, &readSet)) {
        char buffer[BUFFER_SIZE];

        sockaddr_in clientAddr{};
        socklen_t clientAddrLen = sizeof(clientAddr);

        ssize_t bytesReceived = recvfrom(
            serverSocket,
            buffer,
            BUFFER_SIZE - 1,
            0,
            reinterpret_cast<sockaddr*>(&clientAddr),
            &clientAddrLen
        );

        if (bytesReceived <= 0)
            continue;

        buffer[bytesReceived] = '\0';

        string data(buffer, bytesReceived);

        map<char, int> frequencies;

        for (char c : data)
            frequencies[c]++;

        if (frequencies.size() < 3) {
            string message =
                "Number of different characters is less than 3. Discon-
nect.";

            sendto(
                serverSocket,
                message.c_str(),
                message.size(),
                0,
                reinterpret_cast<sockaddr*>(&clientAddr),
                clientAddrLen
            );

            continue;
        }
    }
}

```

```

string response = "<";
bool first = true;

for (const auto& item : frequencies) {
    if (!first)
        response += ", ";

    response += item.first;
    response += "-";
    response += to_string(item.second);

    first = false;
}

response += ">";

sendto(
    serverSocket,
    response.c_str(),
    response.size(),
    0,
    reinterpret_cast<sockaddr*>(&clientAddr),
    clientAddrLen
);
}

if (FD_ISSET(STDIN_FILENO, &readSet)) {
    string command;
    getline(cin, command);

    string keyword;
    string ip;
    int port;

    size_t firstSpace = command.find(' ');
    size_t secondSpace = command.find(' ', firstSpace + 1);

    if (firstSpace != string::npos &&
        secondSpace != string::npos) {

        keyword = command.substr(0, firstSpace);
        ip = command.substr(
            firstSpace + 1,
            secondSpace - firstSpace - 1
        );

        port = stoi(command.substr(secondSpace + 1));

        if (keyword == "disconnect") {
            sockaddr_in clientAddr{};

            clientAddr.sin_family = AF_INET;
            clientAddr.sin_port = htons(port);
            inet_pton(
                AF_INET,
                ip.c_str(),
                &clientAddr.sin_addr
            );

            string message = "Disconnected by server.";

            sendto(
                serverSocket,

```

```

        message.c_str(),
        message.size(),
        0,
        reinterpret_cast<sockaddr*>(&clientAddr),
        sizeof(clientAddr)
    );
    }
}

close(serverSocket);

```

Задание для реализации UDP-клиента:

Вариант	Ввод символов с отсылкой введенной строки по нажатию на клавишу	Ведение файла протокола событий, включающих*: 1) время начала и окончания соединения; 2) передаваемую серверу строку и время передачи строки; 3) принимаемую от сервера строку и время приема строки.	Возможность разрыва соединения при помощи команды: disconnect <адрес> <порт>	1) Задание в программе клиента специальной команды и параметров: подключения к серверу: connect <адрес> <порт> 2) Автоматическое подключение к серверу с заданным по умолчанию адресом при запуске клиента
1	Home	1), 2)	+	2)

Код программы

```

#include <iostream>
#include <fstream>
#include <string>
#include <ctime>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <unistd.h>
#include <termios.h>

using namespace std;

const char* SERVER_IP = "127.0.0.1";
const int SERVER_PORT = 666;
const int BUFFER_SIZE = 1024;

string getTime() {
    time_t now = time(nullptr);

    char buffer[64];

    strftime(

```

```

        buffer,
        sizeof(buffer),
        "%Y-%m-%d %H:%M:%S",
        localtime(&now)
    );

    return buffer;
}

int main() {
    int clientSocket = socket(AF_INET, SOCK_DGRAM, 0);

    sockaddr_in serverAddr{};
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(SERVER_PORT);

    inet_pton(
        AF_INET,
        SERVER_IP,
        &serverAddr.sin_addr
    );

    ofstream logFile("client.log", ios::app);

    logFile << "Start: " << getTime() << endl;

    termios oldTermios;
    termios newTermios;

    tcgetattr(STDIN_FILENO, &oldTermios);

    newTermios = oldTermios;
    newTermios.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(
        STDIN_FILENO,
        TCSANOW,
        &newTermios
    );

    string input;

    cout << "Enter text. Press Home to send:" << endl;

    while (true) {
        char c;
        read(STDIN_FILENO, &c, 1);

        if (c == 27) {
            char c2;
            char c3;

            read(STDIN_FILENO, &c2, 1);
            read(STDIN_FILENO, &c3, 1);

            if ((c2 == '[' || c2 == 'O') && c3 == 'H') {
                cout << endl;

                sendto(

```

```

        clientSocket,
        input.c_str(),
        input.size(),
        0,
        reinterpret_cast<sockaddr*>(&serverAddr),
        sizeof(serverAddr)
    );

    logFile
        << "Sent: "
        << input
        << " | "
        << getTime()
        << endl;

    char buffer[BUFFER_SIZE];

    sockaddr_in senderAddr{};
    socklen_t senderAddrLen = sizeof(senderAddr);

    ssize_t bytesReceived = recvfrom(
        clientSocket,
        buffer,
        BUFFER_SIZE - 1,
        0,
        reinterpret_cast<sockaddr*>(&senderAddr),
        &senderAddrLen
    );

    buffer[bytesReceived] = '\0';

    string response(buffer);

    cout << "Server: " << response << endl;

    if (response.find("Disconnect") != string::npos ||
        response.find("Disconnected") != string::npos) {
        break;
    }

    input.clear();

    cout << "Enter text. Press Home to send:" << endl;
}

continue;
}

if (c == 127 && !input.empty()) {
    input.pop_back();

    cout << "\b \b" << flush;

    continue;
}

input += c;

cout << c << flush;

```

```
}

tcsetattr(
    STDIN_FILENO,
    TCSANOW,
    &oldTermios
);

logFile << "End: " << getTime() << endl;

logFile.close();

close(clientSocket);}
```

Вывод: Приобрел навыки по практическому использованию библиотеки для реализации сетевых приложений на базе протоколов TCP и UDP